

CHAPTER 1

Introduction to PL/pgSQL

In this chapter, we will start with an introduction of PL/pgSQL, on what is PL/pgSQL and what are the key features of it. We will talk about some common use cases where PL/pgSQL is used. PL/pgSQL comes by default when you install the PostgreSQL server. However, we will provide the steps to install PL/pgSQL. We will explain how PL/pgSQL works with a simple flow diagram. We will show some basic examples of PL/pgSQL code blocks which are called anonymous and named code blocks.

A Closer Look at PL/pgSQL

PostgreSQL uses SQL (Structured Query Language) as a default query language. SQL is a common domain-specific language for relational databases. PostgreSQL uses some extensions and features to implement the standards of SQL. In addition to SQL, PostgreSQL supports many procedural languages like PL/pgSQL, PL/Java, PLV8, PL/Python, PL/Perl, etc. Using these languages, you can create functions, stored procedures, and triggers which will improve the performance by reducing the multiple iterations to the databases.

PL/pgSQL is the most commonly used procedural language in PostgreSQL. It is an extension of SQL. It is similar to Oracle's PL/SQL and supports features like control structures, exception handling, variables, loops, and conditional statements. These features help us to develop complex database applications in an efficient way.

When working on designing a complex business logic inside the database, you would need to develop multiple SQLs which are sometimes interdependent. Results of one SQL will be used by other SQLs. In this case, running multiple SQLs increases the data flow between the database and the client application and will cause performance bottlenecks due to high data transfer through the network. To overcome this, you can use stored procedures or functions.

PL/pgSQL supports stored procedures, functions, and triggers. A stored procedure is a set of precompiled SQL statements which can be executed repeatedly. Stored procedures can help to reduce network traffic and improve performance by reducing the amount of data that needs to be sent between the database and the client application.

The common use cases to use stored procedures or functions using PL/pgSQL are

1. Improve data processing speed by using precompiled code through stored procedures which will be faster than raw SQL queries.
2. Write more complex code using features like control structures, exception handling, variables, loops, conditional statements, etc.
3. Using stored procedures or functions, you can create a reusable code to call from the applications to save time and effort.
4. PL/pgSQL is portable across different operating systems and platforms. This makes it easier to migrate code between different environments.
5. Prevent unauthorized access and data breaches by controlling the user authentication on stored procedures or functions.
6. Use triggers to implement constraints of business processes that cannot be expressed as foreign keys or check constraints.

PL/pgSQL Installation

PL/pgSQL is already included in PostgreSQL, so if you have PostgreSQL installed, you should have PL/pgSQL as well. However, you may need to enable it if it is not already enabled. Here are the steps to enable PL/pgSQL in PostgreSQL:

1. Install PostgreSQL psql client to connect to the database, or you can use the pgAdmin client tool.

For Ubuntu, the following are the simple steps to install the client:

Create the file repository configuration:

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt
$(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'
```

Import the repository signing key:

```
wget --quiet -O - https://www.postgresql.org/media/keys/
ACCC4CF8.asc | sudo apt-key add -
```

Update the package lists:

```
sudo apt-get update
```

Install the latest version of PostgreSQL. If you want a specific version, use 'postgresql-12' or similar instead of 'postgresql':

```
sudo apt-get -y install postgresql-client-15
```

For Linux (RHEL), you can follow the steps here:

www.postgresql.org/download/linux/redhat/

2. Connect to the database and check if PL/pgSQL is already installed:

```
postgres=# \dx
List of installed extensions
Name | Version | Schema | Description
-----+-----+-----+-----
(0 rows)

postgres=# select * from pg_extension where extname='plpgsql';
 oid | extname | extowner | extnamespace | extrelocatable | extversion | extconfig | extcondition
-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)
```

3. Execute the following command to enable PL/pgSQL:

```
postgres=# CREATE EXTENSION plpgsql;
CREATE EXTENSION
```

4. Verify that PL/pgSQL is enabled by executing the following command:

```
postgres=# \dx
List of installed extensions
Name      | Version | Schema | Description
-----+-----+-----+-----
plpgsql   | 1.0     | pg_catalog | PL/pgSQL procedural language
(1 row)

JavaScript

postgres=# select * from pg_extension where extname='plpgsql';
 oid | extname | extowner | extnamespace | extrelocatable | extversion | extconfig | extcondition
-----+-----+-----+-----+-----+-----+-----+-----
16388 | plpgsql |      10 |          11 | f               | 1.0        |           |
(1 row)
```

PL/pgSQL Execution Flow

PL/pgSQL is like every other “loadable, procedural language.” PL/pgSQL gets loaded through a function manager called fmgr. The fmgr loads the language handler when a procedural language function or procedure is executed and calls it. The execution flow of PL/pgSQL code is similar to that of other procedural programming languages, with parsing, compilation, execution, and cleanup stages. However, PL/pgSQL code is executed on the server side, which means that it has direct access to the database and can perform database operations more efficiently than client-side code.

On the first call of a PL/pgSQL function or procedure in a session, the server first parses the code to check for syntax errors. The call handler will “compile” a function statement tree once the code is parsed. When the code is compiled, it turns into an internal form that the server can execute more efficiently. SQL queries in the function are just kept as a string at this point, and the expressions like the following are actually SQL queries:

```
my_var := some_param * 10
```

The SQL queries are actually parsed at this point, and parser hooks are used to replace variables/parameters with PARAM nodes in the parse tree. The PL/pgSQL statement tree is very similar to a PostgreSQL execution tree. After the parse and compile, the call handler then executes that statement tree. On the first execution of a statement node that has an SQL query in it, that query is prepared via the Server Programming Interface (SPI). The SPI provides a simple and efficient way to execute SQL commands, retrieve query results, and manipulate the database. The compiled code is then executed by the server. Based on any variable and control structure declaration, the server creates a new execution environment for the PL/pgSQL code. If the PL/pgSQL code is a function or stored procedure that returns a result set, the server will send the result set back to the client. Once the execution of the code is complete, the server will clean up any resources that were used by the PL/pgSQL code, including variables and any temporary tables that were created.

Figure 1-1 represents the flow of execution.

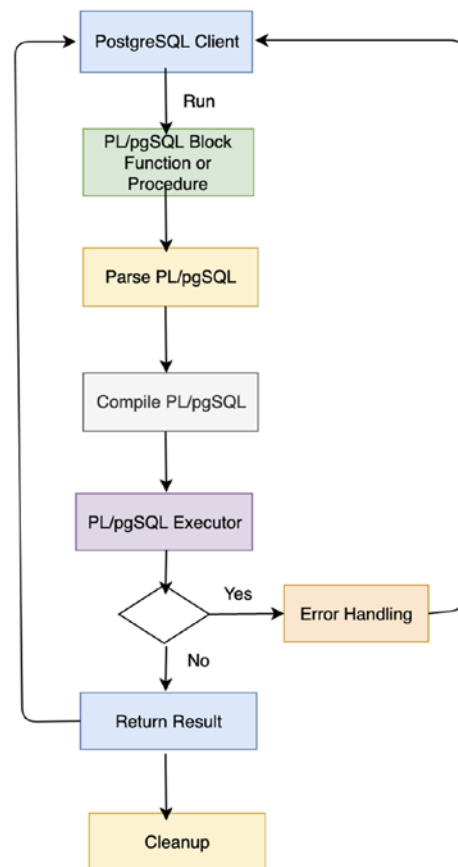


Figure 1-1. *PL/pgSQL execution flow*

This diagram illustrates the high-level steps of the PL/pgSQL execution flow. However, it's important to note that PL/pgSQL code can be quite complex and may include multiple control structures, error handling blocks, and nested and even recursive PL/pgSQL function calls and trigger invocations and database operations. The actual execution flow of a specific PL/pgSQL function or stored procedure will depend on the specific code and logic used. This call hierarchy is not limited to PL/pgSQL. All procedural languages share the common entry point of the `fmgr`, so they can be mixed and matched in trigger and function call stacks.

PL/pgSQL Blocks

PL/pgSQL is a block-structured language. The basic unit in any PL/pgSQL code is a block. All PL/pgSQL code is composed of a single block or blocks that occur either sequentially or nested within another block. There are two kinds of blocks:

- Anonymous or unnamed blocks (DO)
- Named blocks (functions)

Anonymous or Unnamed Blocks

Anonymous or unnamed blocks are generally constructed dynamically and executed only once by the user. It is sort of a complex SQL statement.

The following is the structure of an anonymous block, for example:

```
DO $$
[ <<label>> ]
[ DECLARE
-- Variable declaration here
]
BEGIN
-- Execute statements here
END [ label ];
$$;
```

Now, let us start with a simple hello world code block, which does not have any name associated with it:

```
postgres=# DO
$$
BEGIN
    RAISE NOTICE 'Hello World';
END;
$$;

NOTICE:  Hello World
DO
```

In the preceding example, the `RAISE NOTICE` command will help us to print the given message on the client console. As you can see here, the block is declared without a name, and if you want to print `Hello World`, then you have to repeat the same set of instructions again.

Now, let us print the `Hello World` line by line rather than in a single line:

```
postgres=# DO
$o$
BEGIN
    RAISE NOTICE $i$
    Hello
    World
    $i$;
END;
$o$;

NOTICE:
    Hello
    World
DO
```

In the preceding example, we used different multiline specifiers. The whole block got enclosed by `$o$`, and the inner `Hello World` got enclosed by `$i$`. From this example, we can learn that in PL/pgSQL, we can have the nested multiliners, where each multiline should follow its own enclosure.

Now, let us write a nested BEGIN ... END inside a main BEGIN ... END block. Here is an example:

```
postgres=# DO
$$
BEGIN
    BEGIN
        RAISE NOTICE 'Hello World';
    END;
END;
$$;
NOTICE: Hello World
DO
```

In the preceding example, we print the Hello World message from the nested BEGIN ... END block. It is possible to have multiple nested statements inside a single BEGIN ... END block. We will see more of these in the coming chapters, where we discuss exception handling.

Now, let us go a little deeper and print the Hello World message from the nested unnamed code block:

```
postgres=# DO
$o$
BEGIN
    DO
        $i$
            BEGIN
                RAISE NOTICE 'Hello World';
            END;
        $i$;
    END;
$o$;
NOTICE: Hello World
DO
```

As you can see in the preceding example, we are able to define an unnamed block inside an unnamed block. By defining the nested code blocks, we can segregate a large unnamed block into multiple stand-alone work units. We don't need to write a nested

unnamed block; in most of the cases, the nested BEGIN...END block would be sufficient. But in general, we don't keep large unnamed blocks in the database; rather, we store them inside with a name (function/procedure), and we call that object name, whenever it is required.

Here is another example where we can have a nested block inside an exception:

```
postgres=# DO $inline$
BEGIN
    PERFORM 1/0;
    RAISE NOTICE 'Hello World!';

EXCEPTION
    WHEN OTHERS THEN
    DO $$
    BEGIN
        RAISE NOTICE 'Got error';
    END;
    $$;
END;
$inline$;
NOTICE: Got error
DO
```

PL/pgSQL does not restrict the anonymous blocks as stand-alone objects; we can also embed these inline definitions inside the function or procedure object. We haven't discussed about procedures and functions yet, but showing you an example where you can declare the inline anonymous block inside a function:

```
postgres=# CREATE OR REPLACE FUNCTION test_func()
RETURNS void
LANGUAGE plpgsql
AS $function$
BEGIN
    DO $$ BEGIN RAISE NOTICE 'Hello World!'; END;$$;
END;
$function$;
CREATE FUNCTION
```

```
postgres=# SELECT test_func();
NOTICE:  Hello World!
 test_func
-----
(1 row)
```

How it works is, unlike other procedural languages, PL/pgSQL gives an inline statement handler `plpgsql_inline_handler`. By using this handler, PL/pgSQL executes the unnamed or inline PL/pgSQL blocks. If there are any nested code blocks, then those will be evaluated recursively by the `plpgsql_inline_handler`.

💡 Note that it is not possible to return any value from the unnamed code blocks. Always use anonymous code blocks to define the business logic, which involves making a set of function or procedure calls. If you want to return any value from anonymous blocks, then we might need to use any session-level variables, which need to be set inside the anonymous block, and access them from the outside of the blocks.

Named Blocks

Named blocks have a name associated with them, are stored in the database, can be executed repeatedly, and can take in parameters.

A named block in PL/pgSQL is defined using the following syntax:

```
<<label>>
DECLARE
-- declare variables here
BEGIN
-- Named block's code here
END;
```

Here, `label` is the name of the block and is enclosed within `<< >>` brackets. It is not just cosmetic, but that nested code blocks can refer to outer variables by using that label instead of finding the innermost match for the variable name.

The **DECLARE** section is used to declare variables that are used within the block, while the **BEGIN** and **END** sections contain the actual code for the block.

Once a named block has been defined, it can be called from within the same function or procedure using the **PERFORM** statement:

```
PERFORM block_name;
```

This will execute the code within the named block. Named blocks can be called multiple times within the same function or procedure, allowing for reusable and modular code.

Here's an example of a PL/pgSQL function that uses named blocks to calculate the factorial of a number:

```
CREATE OR REPLACE FUNCTION factorial(num INTEGER)
RETURNS INTEGER AS $$
DECLARE
    result INTEGER := 1;
BEGIN
    <<factorial_loop>>
    FOR i IN 1..num LOOP
        result := result * i;
    END LOOP factorial_loop;

    RETURN result;
END;
$$ LANGUAGE plpgsql;
```

In this example, the named block **factorial_loop** is used within a **FOR** loop to calculate the factorial of the input number. The **DECLARE** section declares a variable **result** to store the final result, while the **BEGIN** and **END** sections contain the code for the named block.

The named block is called within the **FOR** loop using the **LOOP** statement. This allows the loop to continue until it reaches the specified number of iterations.

Once the **FOR** loop is complete, the final result is returned by the function.

To call the function and calculate the factorial of a number, you would execute the following SQL statement:

```
SELECT factorial(5);
```

This would return the value **120**, which is the factorial of 5.

Summary

In this chapter, we talked about PL/pgSQL use cases, installation, and how it works with a flow diagram. We have also shown how simple PL/pgSQL code blocks look like and how to execute them. These examples will help you to understand and start with PL/pgSQL code. In the next chapter, we will talk about the variables that are used inside PL/pgSQL code blocks. We will start with how to declare those variables and dive deep into different types of methods to use based on the use cases. These will help you to decide which type of variables you should use when building the PL/pgSQL code for the functions or procedures.

What's Next

In the next chapter, we will be covering some key features of PL/pgSQL variables like the following:

- **Variable Types:** Explore the different types of variables and learn how to choose the appropriate variable type for your needs.
- **Variable Scoping Mastery:** Gain a better grasp of variable scoping rules and how to manage variables effectively within different blocks.
- **Variable Naming Conventions:** Learn about naming conventions that can help you write more maintainable and readable code.
- **Advanced Variable Usage:** Extend your knowledge by using variables in more complex scenarios.

CHAPTER 2

PL/pgSQL Variables

In the previous chapter, we talked about what PL/pgSQL is and some use cases where you need it. We also explained the steps to install and the execution flow of PL/pgSQL with a simple diagram. We have discussed some PL/pgSQL anonymous and named code block examples which will help in understanding the basics. In this chapter, we will introduce variables that are used in the PL/pgSQL code. We will provide different types of variables and use cases on when to use them. We will show the declaration and scope of variables and different types of variables with some examples.

What Are Variables in PL/pgSQL?

Similar to other programming languages, PL/pgSQL has variables which are used to store values for later use in the program. These variables can hold different types of data such as integers, floating-point numbers, strings, and boolean values.

Declaring Variables

PL/pgSQL offers to declare variables in its declaration section of the block. Here is an example:

```
postgres=# DO
$$
DECLARE
    v_var1 INT;
    v_var2 INT:=10;
BEGIN
    RAISE NOTICE 'v_var1 %', v_var1;
    RAISE NOTICE 'v_var2 %', v_var2;
```

```

END;
$$;
NOTICE:  v_var1 <NULL>
NOTICE:  v_var2 10
DO

```

In the preceding example, we have declared two variables. The `v_var1` variable is declared but not defined with any value to it. The second variable `v_var2` is declared but has a value 10 to it. If we try to access a variable which is declared and not defined, then expect that we would get NULL from it. In the preceding output, we can also see that the `v_var1` is set to NULL.

From the previous chapter, we learned that we can have nested blocks inside the main PL/pgSQL block. By using that nested block feature, we have multiple declarations and multiple variables in the single PL/pgSQL block. Here is an example:

```

postgres=# DO
$o$
DECLARE
    v_var1 INT:=10;
BEGIN
    RAISE NOTICE 'v_var1 %', v_var1;

    DO
    $i$
    DECLARE
        v_var1 INT:=100;
    BEGIN
        RAISE NOTICE 'v_var1 %', v_var1;
    END;
    $i$;

END;
$o$;
NOTICE:  v_var1 10
NOTICE:  v_var1 100
DO

```

In the preceding example, we have nested unnamed blocks, where we have two variable declarations. The variable `v_var1` in the main block is declared with the value 10, and also `v_var1` in the inside block is declared with the value 100. Also, the scope of the variable is always local.

Note In PL/pgSQL, we can't declare variables on the fly. We should always declare them in the `DECLARE` section, and we have to use them in the block. But, while defining the `FOR LOOP`, we can have the variables declared on the fly. We will discuss more about this in the coming chapters.

Variable Scope

The scope of the declared variables in PL/pgSQL is always local to its current block. That is, once we declare the variable in a block, we can't access them outside of that block. Here is an example:

```
postgres=# DO
$$
DECLARE
BEGIN
    DECLARE
        v_var1 INT:=10;
    BEGIN
        RAISE NOTICE 'v_var1 %', v_var1;
    END;
    RAISE NOTICE 'v_var1 %', v_var1;
END;
$$;
NOTICE:  v_var1 10
ERROR:  column "v_var1" does not exist
LINE 1: v_var1
        ^

QUERY:  v_var1
CONTEXT:  PL/pgSQL function inline_code_block line 10 at RAISE
```

From the preceding example, as you can see we got an error, column "v_var1" does not exist, when we try to access the variable v_var1 from the outer BEGIN...END block. That is, the scope of variable v_var1 is local to that inner BEGIN...END block, and we can't access them from the outside. If we declare the variable v_var1 in the parent BEGIN...END block, then we can access that variable inside the nested BEGIN...END blocks too. Because the v_var1 is declared at the parent block level, its scope is at the whole block level. Now, you might have questions like what if we declare the same variable v_var1 in parent and nested BEGIN...END blocks and how to access the parent block's v_var1 along with the local declared variable. Here is an example for this use case:

```
postgres=# DO
$$
DECLARE
    v_var1 INT:=1;
BEGIN
    DECLARE
        v_var1 INT:=10;
    BEGIN
        RAISE NOTICE 'v_var1 %', v_var1;
    END;
END;
$$;
NOTICE:  v_var1 10
DO
```

From the preceding example, we were only able to access the variable v_var1, which was declared in the nested BEGIN...END block. To access the parent v_var1 variable, then we should access that variable with the block's label. That is, we have to give a label name to the parent block, and then we should access the v_var1 along with its label name. Here is an example:

```
postgres=# DO
$$
<<parent>>
DECLARE
    v_var1 INT := 1;
```

```

BEGIN
    DECLARE
        v_var1 INT := 10;
    BEGIN
        RAISE NOTICE 'Parent v_var1 %', parent.v_var1;
        RAISE NOTICE 'Local v_var1 %', v_var1;
    END;
END;
$$;
NOTICE: Parent v_var1 1
NOTICE: Local v_var1 10
DO

```

Constant Variables

We can declare constant variables inside PL/pgSQL, which shouldn't get updated by further instructions. Here is an example:

```

postgres=# DO
$$
DECLARE
    v_c_pi CONSTANT REAL DEFAULT 3.14;
BEGIN
    v_c_pi = 3.15;
END;
$$;
ERROR: variable "v_c_pi" is declared CONSTANT
LINE 6:     v_c_pi = 3.15;

```

In the preceding example, we declared the variable `v_c_pi` as `CONSTANT` and set its `DEFAULT` value as `3.14`. But, in further instructions, when we tried to update its value as `3.15`, we got the exception as the variable is declared as `CONSTANT`, which should not get updated by any of the instructions.

Variable Alias

In PL/pgSQL, we can also create a reference variable which points to another variable or system variables. For example, if we want to create a reference variable or a short-length variable name to another variable, then we can create those short-length variables using ALIAS. Here is an example:

```
DO
$$
DECLARE
    var_earth_sun_distance REAL DEFAULT 149.6;
    v_e_s_d ALIAS FOR var_earth_sun_distance;
BEGIN
    RAISE NOTICE 'Reference      #1 %', v_e_s_d;
    v_e_s_d = 149.5;
    RAISE NOTICE 'Actual Variable  %', var_earth_sun_distance;
END;
$$;
NOTICE:  Reference      #1 149.6
NOTICE:  Actual Variable  149.5
DO
```

In the preceding example, we created a reference variable `v_e_s_d` to the actual variable `var_earth_sun_distance`. Also, if we perform any update on the reference variable, then we can see those changes from the actual variable. ALIAS is not limited to creating a reference variable to the actual variable, it will allow creating a reference to another reference. Here is an example:

```
DO
$$
DECLARE
    var_earth_sun_distance REAL DEFAULT 149.6;
    v_e_s_d ALIAS FOR var_earth_sun_distance;
    vd ALIAS FOR v_e_s_d;
BEGIN
    RAISE NOTICE 'Reference      #1 %', v_e_s_d;
    RAISE NOTICE 'Reference      #2 %', vd;
```

```

RAISE NOTICE 'Update Ref #1';
v_e_s_d = 149.5;
RAISE NOTICE 'Actual Variable    %', var_earth_sun_distance;
RAISE NOTICE 'Update Ref #2';
vd = 149.4;
RAISE NOTICE 'Actual Variable    %', var_earth_sun_distance;
END;
$$;
NOTICE:  Reference          #1 149.6
NOTICE:  Reference          #2 149.6
NOTICE:  Update Ref #1
NOTICE:  Actual Variable    149.5
NOTICE:  Update Ref #2
NOTICE:  Actual Variable    149.4
DO

```

In the preceding example, we created an ALIAS variable from another ALIAS variable. That is, we created a new reference variable, which points to another reference variable. Also, if we update the second reference variable, it will reflect those changes on the actual variable via the first reference.

PL/pgSQL supports the following types of variables:

- Scalar Variables
- Array Variables
- Record Variables
- Cursor Variables

Scalar Variables

In all the previous examples, we demonstrated scalar variables. Scalar variables hold a single value of a specific data type, such as an integer or a string. They can be declared and initialized using the DECLARE keyword and can be assigned values using the := assignment operator.

For example, the following code declares each type of scalar variable:

```
postgres=# DO $$
DECLARE
    my_int integer := 1;
    my_text text := 'Hello, world!';
    my_bool boolean := true;
BEGIN
    -- perform operations on the scalar variables
    my_int := my_int + 10;
    my_text := my_text || ' How are you?';
    my_bool := not my_bool;

    -- print the scalar variables
    RAISE NOTICE 'my_int = %', my_int;
    RAISE NOTICE 'my_text = %', my_text;
    RAISE NOTICE 'my_bool = %', my_bool;
END;
$$;
NOTICE: my_int = 11
NOTICE: my_text = Hello, world! How are you?
NOTICE: my_bool = f
DO
```

In the preceding example, we declare three scalar variables: `my_int` with a value of 1 and a data type of integer, `my_text` with a value of 'Hello, world!' and a data type of text, and `my_bool` with a value of true and a data type of boolean. We then perform some operations on these variables using arithmetic, concatenation, and logical negation. Finally, we print the values of the variables using the `RAISE NOTICE` statement.

Scalar variables are useful for storing temporary values or performing calculations within a stored procedure or function. They can be used in a variety of ways, such as tracking state, performing conditional logic, or holding input or output parameters.

Array Variables

Array variables are variables that can hold multiple values of the same data type. They are declared using a data type followed by the `[]` syntax, such as `integer[]` or `text[]`.

For example, the following code declares an array variable:

```
postgres=# DO $$
DECLARE
    my_array integer[] := '{1, 2, 3, 4, 5}';
BEGIN
    -- print the entire array
    RAISE NOTICE 'my_array = %', my_array;

    -- access individual elements of the array
    RAISE NOTICE 'my_array[2] = %', my_array[2];

    -- modify individual elements of the array
    my_array[3] := 10;

    -- print the modified array
    RAISE NOTICE 'my_array = %', my_array;
END;
$$;
NOTICE: my_array = {1,2,3,4,5}
NOTICE: my_array[2] = 2
NOTICE: my_array = {1,2,10,4,5}
DO
```

In the preceding example, we declare an array variable `my_array` with a data type of `integer[]` and initialize it with the values `{1, 2, 3, 4, 5}`. We then print the entire array using the `RAISE NOTICE` statement, access and print the value of the second element of the array, modify the value of the third element of the array, and print the entire array again.

Array variables are useful for storing and manipulating sets of related data, such as lists of numbers, strings, or boolean values. They can be used in a variety of ways, such as for performing calculations on multiple values at once, storing input or output parameters, or passing data between functions or procedures.

Record Variables

Record variables are used to store a row of data from a table or a query result. They are declared using the %ROWTYPE attribute and can be assigned values using the SELECT INTO statement.

For example, the following code declares a record variable:

```
postgres=# CREATE TABLE emp (emp_id INT, emp_name VARCHAR, emp_salary
NUMERIC);
CREATE TABLE

postgres=# INSERT INTO emp VALUES (100, 'SCOTT', '10000.00');
INSERT 0 1

postgres=# DO $$
DECLARE
    my_record emp%ROWTYPE;
BEGIN
    -- select a row of data into the record variable
    SELECT * INTO my_record FROM emp WHERE emp_id = 100;

    -- print the values of the record variable
    RAISE NOTICE 'emp_id = %', my_record.emp_id;
    RAISE NOTICE 'emp_name = %', my_record.emp_name;
    RAISE NOTICE 'emp_salary = %', my_record.emp_salary;

    -- update the values of the record variable
    my_record.emp_name := 'Smith';
    my_record.emp_salary := 50000;

    -- update the row of data in the table
    UPDATE emp SET emp_name = my_record.emp_name, emp_salary = my_record.
emp_salary WHERE emp_id = 1001;
END;
$$;
NOTICE:  emp_id = 100
NOTICE:  emp_name = SCOTT
NOTICE:  emp_salary = 10000.00
DO
```

In the preceding example, we declare a record variable `my_record` that holds a row of data from the `emp` table. We then select a row of data into the record variable using a `SELECT` statement, print the values of the record variable using the `RAISE NOTICE` statement, update the values of the record variable, and update the row of data in the table using an `UPDATE` statement.

Record variables are useful for storing and manipulating rows of data from tables or query results within a stored procedure or function. They can be used in a variety of ways, such as for passing data between functions or procedures, performing calculations on data, or storing input or output parameters.

Cursor Variables

Cursor variables are variables that hold a reference to a cursor, which is a named SQL statement that can be executed repeatedly. They are declared using the `CURSOR` keyword.

For example, the following code declares a record variable named “`my_row`” that corresponds to the columns of a table named “`my_table`”:

```
postgres=# CREATE TABLE emp (emp_id INT, emp_name VARCHAR, emp_salary
NUMERIC);
CREATE TABLE

postgres=# INSERT INTO emp VALUES (100, 'SCOTT', '10000.00');
INSERT 0 1

postgres=# INSERT INTO emp VALUES (100, 'ROBERT', '50000.00');
INSERT 0 1

postgres=# DO $$
DECLARE
    my_cursor refcursor;
    my_record emp%ROWTYPE;
BEGIN
    -- open the cursor and fetch the first row of data
    OPEN my_cursor FOR SELECT * FROM emp;
    FETCH my_cursor INTO my_record;

    -- print the values of the first row of data
```

```

RAISE NOTICE 'emp_id = %', my_record.emp_id;
RAISE NOTICE 'emp_name = %', my_record.emp_name;
RAISE NOTICE 'emp_salary = %', my_record.emp_salary;

-- fetch the next row of data
FETCH my_cursor INTO my_record;

-- print the values of the second row of data
RAISE NOTICE 'emp_id = %', my_record.emp_id;
RAISE NOTICE 'emp_name = %', my_record.emp_name;
RAISE NOTICE 'emp_salary = %', my_record.emp_salary;

-- close the cursor
CLOSE my_cursor;
END;
$$;
NOTICE:  emp_id = 100
NOTICE:  emp_name = SCOTT
NOTICE:  emp_salary = 10000.00
NOTICE:  emp_id = 100
NOTICE:  emp_name = ROBERT
NOTICE:  emp_salary = 50000.00
DO

```

In the preceding example, we declare a cursor variable `my_cursor` and a record variable `my_record` that holds a row of data from the `emp` table. We then open the cursor using a `SELECT` statement, fetch the first row of data into the record variable using a `FETCH` statement, print the values of the first row of data using the `RAISE NOTICE` statement, fetch the second row of data into the record variable, print the values of the second row of data, and then close the cursor using the `CLOSE` statement.

Cursor variables are useful for manipulating and iterating through result sets returned by queries. They can be used in a variety of ways, such as for processing large datasets, performing calculations on data, or storing input or output parameters.

Variables can be very useful in PL/pgSQL programs as they allow you to store and manipulate data in a flexible way.

Summary

In this chapter, we learned about the types of variables with nice examples and explanations of how they work. We went through the declaration of variables in the DECLARE section of the code as well as in an independent code block on the fly. The scope of variables is different for each block of PL/pgSQL code. We explained how the scope of variables varies with some examples. Also, we looked at constant variables and how to use aliases for the variables.

What's Next

In the next chapter, we will be covering some key features of PL/pgSQL data types like the following:

- **Advanced Data Type Exploration:** Delve into complex data types like composite and range types to efficiently manage intricate data structures.
- **Custom Domain Types:** Create user-defined domain types with specific constraints to ensure data integrity and validation in your applications.
- **Optimal Type Selection:** Understand how to choose the right data type based on application needs, considering performance, maintainability, and space efficiency.
- **Dynamic Application Building:** Harness PL/pgSQL data types to create adaptable applications that handle diverse data inputs and outputs.

CHAPTER 3

PL/pgSQL Data Types

In the previous chapter, we introduced variables that are used in the PL/pgSQL code. We talked about different types of variables and use cases on when to use them. We have shown the declaration and scope of the variables and different types of variables with some examples. In this chapter, we will introduce the supported data types to use in PL/pgSQL code. We will walk through each data type and use cases with some examples.

Data Types

Choosing the right data type for the data being used is a best practice that developers should follow. Choosing the correct data type can help us avoid many data validation checks and prevent exceptions when incorrect data is entered. The PostgreSQL database engine supports multiple data types for each data kind. By using these data types, we build tables and insert proper data into them. We can use most of the PostgreSQL data types inside PL/pgSQL blocks, besides pseudo-types like `any`, `anyenum`, and `record`. It is not just limited to using existing data types; we can also construct new composite types by using these existing data types.

Understanding data types is crucial for developing efficient and effective database applications. We will cover data types such as integers, floating-point numbers, booleans, and strings, as well as more advanced data types like arrays and composite types. By the end of this chapter, you will have a good understanding of the different data types available in PL/pgSQL and how to use them effectively in your business logic. We will also cover how to declare variables and constants using these data types and how to convert between different data types using type casting. This knowledge is essential for writing robust and error-free PL/pgSQL code that can handle a variety of data inputs and outputs.

Note You can find the list of supported data types here: www.postgresql.org/docs/current/datatype.html.

Declaring Variables with Data Types

In the previous chapter, we saw several examples where we declared variables along with their data types. Now, let us begin with a simple example where we demonstrate the primitive data types like `int`, `boolean`, and `text`:

```
postgres=# DO
$$
declare
v_int int:=10;
v_text text:='text';
v_boolean boolean:=false;
begin
raise notice 'v_int      %',v_int;
raise notice 'v_text     %',v_text;
raise notice 'v_boolean  %',v_boolean;
end;
$$;

NOTICE:  v_int      10
NOTICE:  v_text     text
NOTICE:  v_boolean  f
DO
```

In the preceding example, we declared three variables along with their data types and printed the results. Let us rewrite the previous example as follows and see the results:

```
postgres=# DO
$$
declare
int int:=10;
text text:='text';
```

```

boolean boolean:=false;
begin
raise notice 'int %',int;
raise notice 'text %',text;
raise notice 'boolean %', boolean;
end;
$$;
NOTICE:  int  10
NOTICE:  text text
NOTICE:  boolean f
DO

```

As you can see in the preceding example, variable names can be declared with the corresponding data type names. As data type names are not reserved keywords, we can also use them as variable names. Here, we are demonstrating the possibility of declaring variable names as data types. However, this kind of variable declaration is not typically used in production.

In the previous example, as well as in the previous chapter, we discussed only the base scalar data types in PostgreSQL. These data types accept only a single value into their declared variable type. To obtain a list of available base scalar types in the current database, we can query the PostgreSQL catalog table `pg_type`. The following query returns base data types in the database limited to three only:

```

postgres=# SELECT typename FROM pg_type WHERE typtype='b' LIMIT 3;
typename
-----
bool
bytea
char
(3 rows)

```

Supported Types

PostgreSQL not only supports base types but also composite types, enumerated types, pseudo-types, and range types. To list all the supported type categories in the database, query the `pg_type` catalog table with the following query:

```
postgres=# SELECT distinct typtype::char FROM pg_type;
 typtype
-----
 r
 m
 p
 d
 c
 b
(6 rows)
```

In the preceding output we see, the return values are m, r, p, d, c, and b. All the types, including the user-defined types, will fall under these category types. Table [3-1](#) describes each code value.

Table 3-1. Code values

Code	Description	Example
r	Range type	postgres=# SELECT daterange(current_date, current_date+30);daterange-----[2023-03-15,2023-04-14)(1 row)
m	Multirange type	postgres=# SELECT daterange(daterange(current_date, current_date+30), daterange(current_date+31, current_date+60));datemultirange-----{[2023-03-15,2023-04-14), [2023-04-15,2023-05-14)}(1 row)
p	Pseudo-type	postgres=# SELECT (1,2,3)::anyelement;row------(1,2,3)(1 row)
d	Domain type	postgres=# SELECT 'YES'::information_schema.yes_or_no;yes_or_no-----YES(1 row)
c	Composite type	Any row type or user defined type postgres=# SELECT ('port', '5432')::record::pg_config;row------(port,5432)(1 row)
b	Base type	postgres=# SELECT 1::int;int4-----1(1 row)

Base Type

Base types are the primitive building blocks for other data types in the database. For example, take the domain type `information_schema.yes_or_no` which only accepts the text which is of length 3, and the values have to be YES or NO. This domain type is used as a replacement to `bool` to maintain the `information_schema` standards. Let us examine the underlying data type of this `yes_or_no` domain type using the `format_type` function as follows:

```
postgres=# SELECT format_type(typtype, typtypmod), typtype FROM
pg_type WHERE typname='yes_or_no';
      format_type      | typtype
-----+-----
character varying(3) |      1043
(1 row)
```

From the preceding output, we got the domain's base type as 1043. By using the following query, we see what is the data type which is mapped to 1043:

```
postgres=# SELECT 1043::regtype::text;
      text
-----
character varying
(1 row)
```

By using the following query, see whether this 1043 is a base type or not:

```
postgres=# SELECT typtype, typname FROM pg_type WHERE oid=1043;
 typtype | typname
-----+-----
b       | varchar
(1 row)
```

As you can see from the preceding results, the base types are the primitive building blocks to the data types which we create. By using these base types, we can build our own composite, domain, and range types.

Composite Type

In PostgreSQL, it is possible to create custom data types using the base data types. Custom data types can be defined using not only the base types but also other custom data types or other data types excluding pseudo-types.

Let us construct a simple composite type with the help of base types:

```
postgres=# CREATE TYPE all_fields_base AS (a int, b text, c date);
CREATE TYPE
```

```
postgres=# \d all_fields_base
```

```
Composite type "public.all_fields_base"
```

Column	Type	Collation	Nullable	Default
a	integer			
b	text			
c	date			

Here, we created a type `all_fields_base` as its member fields declared as base data types. Now, write a simple PL/SQL block, which uses this composite as one of its variable's desired data types:

```
postgres=# DO
$$
DECLARE
    v_composite all_fields_base;
BEGIN
    v_composite:= (1, 't', current_date)::all_fields_base;
    RAISE NOTICE '%', v_composite;
END;
$$;
NOTICE:  (1,t,2023-03-18)
DO
```

As you can see in the preceding example, we created the variable `v_composite` with data type `all_fields_base` and assigned multiple values `(1, 't', current_date)` as a single unit. Here, we also used `::` which is a data type casting operator. That is, this `::` operator converts the provided values into the defined data type.

It is not just limited to use only the base types as members inside the composite type. We can also declare a member as a composite, pseudo, or range type. Consider the following example:

```
postgres=# CREATE TYPE mixed_fields_type AS (d real, f all_fields_base);
CREATE TYPE
postgres=# \d mixed_fields_type
```

Composite type "public.mixed_fields_type"

Column	Type	Collation	Nullable	Default
d	real			
f	all_fields_base			

Now, let's write a simple example by using the preceding `mixed_fields_type` data type and assign some value to it:

```
DO
$$
DECLARE
    v_mixed_type mixed_fields_type;
BEGIN
    v_mixed_type:= (3.14, (1, 't', current_date)::all_fields_
base)::mixed_fields_type;
    RAISE NOTICE '%', v_mixed_type;
    RAISE NOTICE 'Value % Type %', v_mixed_type.d, pg_typeof(v_mixed_type.d);
    RAISE NOTICE 'Value % Type %', v_mixed_type.f, pg_typeof(v_mixed_type.f);
END;
$$;
NOTICE:  (3.14,(1,t,2023-03-18))
NOTICE:  Value 3.14 Type real
NOTICE:  Value (1,t,2023-03-18) Type all_fields_base
DO
```

From the preceding example, as you can see we saved a nested composite value into the nested composite variable. Also, by using the object reference notation, we are able to access the required field from this composite type. By using `pg_typeof`, we displayed the data type of the member variable.

Domain Type

Domain types are used when a particular set of constraints need to be enforced on a base type. They are essentially a user-defined type that is based on an existing type. We can create a domain type using the `CREATE DOMAIN` statement. A domain type can be used when we need to enforce certain conditions on input values. For example, `information_schema.yes_or_no` is a domain that only allows YES or NO values. Here is an example:

```
postgres=# SELECT 'YES'::information_schema.yes_or_no;
 yes_or_no
-----
 YES
(1 row)
```

```
postgres=# SELECT 'TRUE'::information_schema.yes_or_no;
ERROR:  value for domain information_schema.yes_or_no violates check
constraint "yes_or_no_check"
```

In the preceding example, the domain type `information_schema.yes_or_no` only accepts YES and does not recognize TRUE. This is how the domain type is constructed.

Let's build our own custom domain type called `tiny_text`, which only accepts up to eight characters. If the input size is greater than eight characters, an error should be thrown:

```
postgres=# CREATE DOMAIN tiny_text text CONSTRAINT tiny_text_length_
constraint CHECK (length(VALUE)<=8);
CREATE DOMAIN
postgres=# CREATE TABLE preferences(short_userid tiny_text, short_pass
tiny_text);
CREATE TABLE
postgres=# INSERT INTO preferences VALUES ('a0bef74', 'i89llk');
INSERT 0 1
```

Try inserting text that is more than eight characters and see if the validation is enforced or not:

```
postgres=# INSERT INTO preferences VALUES ('this is big user id', 'this is
big password');
ERROR:  value for domain tiny_text violates check constraint "tiny_text_
length_constraint"
```

In the preceding example, if you attempt to send more than eight characters, the domain check constraint is violated, and an error message will be displayed as shown earlier.

Domain types can also be created on top of composite types. Consider the following example, where we create a common type called `products_type` and then create a domain type, `toys_type`, with a constraint. This type will only allow products in the `toys` category and will throw an error if you try to insert products from another category.

```
postgres=# CREATE TYPE products_type as (id text, category text, sku int,
price real);
CREATE TYPE

postgres=# CREATE DOMAIN toys_type products_type CHECK ((VALUE).
category='toys');
CREATE DOMAIN

postgres=# CREATE TABLE toys_products(t toys_type);
CREATE TABLE

postgres=# INSERT INTO toys_products VALUES (('dfd89', 'toys', 10, 20.2));
INSERT 0 1
```

Now, try to insert another category product and see if the domain constraint got enforced or not:

```
postgres=# INSERT INTO toys_products VALUES (('dfd99', 'books', 15, 15.2));
ERROR:  value for domain toys_type violates check constraint "toys_type_check"
```

Perfect! We can restrict other categories of data from entering the `toys_products` table. We can achieve this with a plain check constraint on the table, but creating a domain type allows us to not only use `toys_type` at the table level but also across procedures and functions. This domain type ensures that the `toys_type` variable always holds toy category data.

Pseudo-Type

PostgreSQL supports several pseudo data types, which have specific uses. We cannot use them directly, but we can use them whenever required. In other words, these types are only for internal use or to give instructions to PostgreSQL whenever it is required. For example, `void` is a pseudo data type that specifies the return data type of a function. It should only be used as a function return type, and not as a table field type. Here is an example:

```
postgres=# CREATE TABLE test_table(v void);
ERROR:  column "v" has pseudo-type void

postgres=# DO
$$
DECLARE
t void;
BEGIN
t:='';
END;
$$;
ERROR:  variable "t" has pseudo-type void
CONTEXT:  compilation of PL/pgSQL function "inline_code_block" near line 3
```

PostgreSQL is not allowed to create a composite type with pseudo-type:

```
postgres=# CREATE TYPE test_type AS (t void);
ERROR:  column "t" has pseudo-type void
```

Like `void`, there are few other pseudo data types as follows:

- The `any` type can be used to present any function/procedure parameter arguments.
- `trigger` is a pseudo data type which we only use for the trigger's function return type.
- `opaque` is a pseudo data type which is used internally or by the external plug-ins.
- `record` is a pseudo data type which in general is used as a function return type.

Range Type

PostgreSQL supports range types, which are designed to handle range values. You can store and manipulate a range of values in a single column.

This data type is useful in situations where you need to store a range of values rather than a single value, such as

Time Ranges: You can use range data types to store time ranges, such as business hours, meeting times, or shifts.

Numeric Ranges: You can use range data types to store ranges of numeric values, such as temperature, age, or income ranges.

Geographic Ranges: You can use range data types to store geographic ranges, such as latitude and longitude ranges, or ranges of distances.

Text Ranges: You can use range data types to store ranges of text values, such as character or string ranges.

For instance, if you want to store “from” and “to” values as a single unit, you can use a range data type to represent such values. As an example, consider a table called `fleet_mileage` that records the mileage of each vehicle using a different type of gasoline. Each vehicle’s odometer reading will be different at the start, and the gasoline will run out at a different odometer reading. We should store both of these values and then calculate the mileage we achieved with the specific type of gasoline we used. Here is an example:

```
postgres=# CREATE TABLE fleet_mileage(gasoline_id int, vehicle_id int,
readings numrange);
CREATE TABLE
postgres=# INSERT INTO fleet_mileage VALUES (1,1,numrange(1455.2,
1567.5,'[]'));
INSERT 0 1
postgres=# INSERT INTO fleet_mileage VALUES (1,2,numrange(1255.2,
1374.5,'[]'));
INSERT 0 1
postgres=# INSERT INTO fleet_mileage VALUES (1,3,numrange(1132.7,
1259.6,'[]'));
INSERT 0 1
```

In the preceding example, we created a table called `fleet_mileage`, with a `readings` column of type `numrange`. This column stores the beginning and end odometer readings as a range, by including both values in that range. The syntax `[]` indicates that the begin and end range values are inclusive within that range.

Query the table to retrieve a list of vehicles sorted in descending order based on their mileage:

```
postgres=# SELECT *,upper(readings)-lower(readings) as mileage FROM fleet_mileage ORDER BY 4 DESC;
```

gasoline_id	vehicle_id	readings	mileage
1	3	[1132.7,1259.6]	126.9
1	2	[1255.2,1374.5]	119.3
1	1	[1455.2,1567.5]	112.3

(3 rows)

In the preceding query, we used range-specific functions like `upper` and `lower`. These functions return the beginning and end of the range values.

PostgreSQL also offers the ability to create custom range types, similar to the `DOMAIN` type. Now, let's consider another use case where we have table reservations in a hotel:

```
postgres=# CREATE TYPE room_reserve_ts_range AS RANGE (SUBTYPE=timestamp);
CREATE TYPE
```

```
postgres=# CREATE TABLE room_bookings(id int, guest_name text, booking_dates room_reserve_ts_range);
CREATE TABLE
```

```
postgres=# INSERT INTO room_bookings VALUES (1, 'Scott', '[2023-03-22 00:00, 2023-03-25 10:00)');
INSERT 0 1
```

```
postgres=# INSERT INTO room_bookings VALUES (2, 'George', '[2023-03-20 00:00, 2023-03-23 10:00)');
INSERT 0 1
```

```
postgres=# INSERT INTO room_bookings VALUES (3, 'Denis', '[2023-03-15 00:00, 2023-03-23 10:00)');
INSERT 0 1
```

In the preceding example, we created a custom type `room_reserve_ts_range` which is a subtype or underlying base type of `timestamp`. The custom type we created is similar to the `tsrange` type natively offered by PostgreSQL. We can also use range operators such as `contains`, `overlap`, and `adjacence` on the `booking_dates` column. For example, to list all the guests who reserved their rooms on 2023-03-21, we can query this table as follows:

```
postgres=# SELECT guest_name FROM room_bookings WHERE booking_dates @>
'2023-03-21'::timestamp;
 guest_name
-----
 George
  Denis
(2 rows)
```

Multirange Types

PostgreSQL supports multirange data type which represents a set of nonoverlapping ranges of values of some other data type. For example, you could define a multirange type that represents a set of nonoverlapping ranges of integers.

The multirange data type in PostgreSQL is useful for storing and querying sets of nonoverlapping ranges of values. It can be used in various applications such as time series analysis, event scheduling, and resource allocation.

Consider the same example table `fleet_mileage` that is used for range data types. For multirange data types, you can use the built-in `nummultirange` data type:

```
postgres=# CREATE TABLE fleet_mileage(gasoline_id int, vehicle_id int,
readings nummultirange);
CREATE TABLE
postgres=# INSERT INTO fleet_mileage VALUES (1,1,nummultirange(numran
ge(1455.2, 1567.5,'[]'),numrange(1655.2, 1767.5,'[]')));
INSERT 0 1
postgres=# INSERT INTO fleet_mileage VALUES (1,1,nummultirange(numran
ge(1255.2, 1367.5,'[]'),numrange(1455.2, 1667.5,'[]')));
INSERT 0 1
```

```
postgres=# INSERT INTO fleet_mileage VALUES (1,1,nummultirange(numran
ge(1155.2, 1267.5,'[]'),numrange(1355.2, 1567.5,'[]')));
INSERT 0 1
```

In the preceding example, the `reading` column has been created as a `multirange` data type, and you can see a couple of range values.

You can query the table to retrieve a list of vehicles sorted in descending order based on their mileage. However, in `multirange`, it will pick up the higher and lower values from any range in `multirange` values.

```
postgres=# SELECT *,upper(readings)-lower(readings) as mileage FROM fleet_
mileage ORDER BY 4 DESC;
```

gasoline_id	vehicle_id	readings	mileage
1	1	{[1255.2,1367.5],[1455.2,1667.5]}	412.3
1	1	{[1155.2,1267.5],[1355.2,1567.5]}	412.3
1	1	{[1455.2,1567.5],[1655.2,1767.5]}	312.3

(3 rows)

Summary

In this chapter, we learned about data types with nice examples and explanations on how they work. We talked about the use cases where you can use a particular data type so that it helps you while designing an application based on the requirements. In the next chapter, we will talk about the control structures that are used in PL/pgSQL programming. We will cover conditional operations and simple loops with use cases and some examples.

What's Next

In the next chapter, we will be covering some key features of PL/pgSQL strings, numbers, and arrays like the following:

- **String Handling:** Understand different string data types (`char(n)`, `varchar(n)`, `text`) and best practices for selection based on data model design.

- **Numeric Operations:** Utilize various numeric data types (smallint, int, bigint, numeric, float, double precision) to store and process numerical data efficiently.
- **Array Usage:** Leverage arrays to work with lists of data, including declaration, initialization, and accessing array elements.
- **String Formatting:** Explore the format() function for dynamic string creation and manipulation and its usage in constructing SQL statements.
- **Data Transformation:** Discover methods to transform and reshape your data using string and array functions for various use cases.
- **Efficiency Techniques:** Explore optimization techniques to efficiently handle strings, numbers, and arrays in your PL/pgSQL code.